

OCR - PROJET 6

**Anticipez les besoins en consommation de
bâtiments**

Mathilde LE SOLLIEC
11-2025

SOMMAIRE

- 1 Contexte
- 2 Analyse et préparation des données
- 3 Modélisation
- 4 Création d'une API
- 5 Déploiement dans le cloud

1. Contexte

Relevés de la consommation énergétique des bâtiments de Seattle

Des relevés ont été effectués par les agents de la ville de SEATTLE en 2016.

Ces relevés sont coûteux à obtenir.

L'objectif est d'estimer pour ceux pour lesquels elles n'ont pas encore été mesurées, à partir de ceux déjà réalisés



Prédire les émissions de CO2 et la consommation totale d'énergie de bâtiments - non destinés à l'habitation



En se basant sur les données structurelles des bâtiments



Créer un API permettant d'interroger le modèle

2. Analyse et préparation des données

Data cleaning et analyse exploratoire

Provenance des données

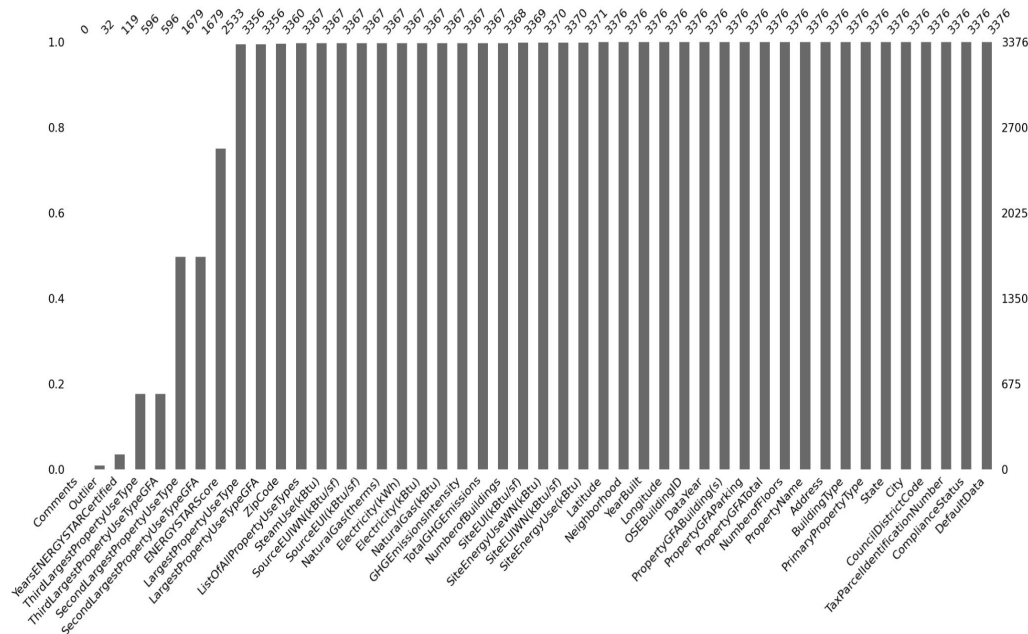
Site de la ville de Seattle

Structure de la BDD

3376 lignes : le nombre total de bâtiments

46 colonnes : le nombre de variables, qui sont les caractéristiques de ces bâtiments.

Peu de valeurs manquantes

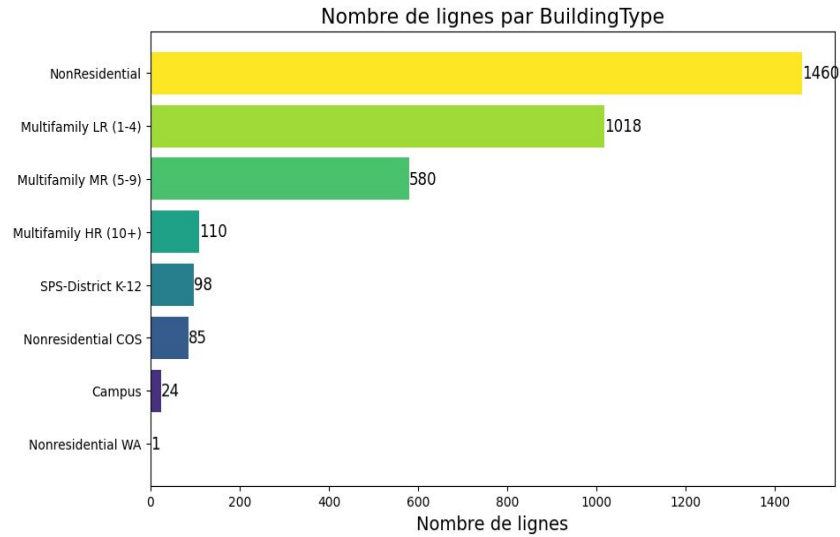


Informations disponibles
Types de bâtiments
Localisation : adresse, latitude, longitude
Année de construction
Nombre de bâtiments et d'étages
Surfaces
Type d'usage et consommations associées
Performances énergétiques - Consommations énergétiques (kBtu, kWh, therms...)
Émissions GES : TotalGHGEmissions, GHGEmissionsIntensity
Variables "autres" : Comments, DefaultData, ComplianceStatus, Outlier

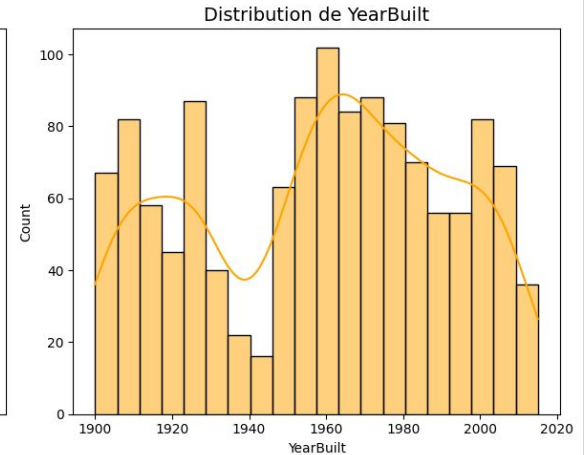
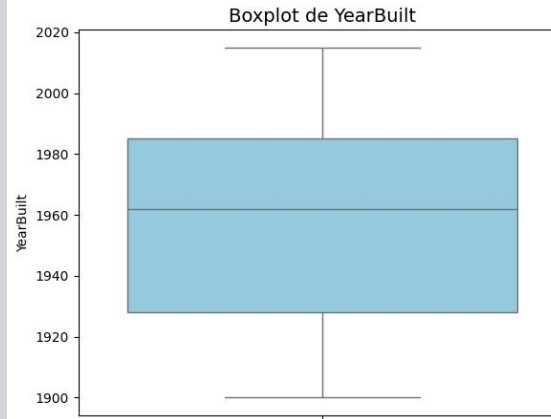
Source et informations :

https://data.seattle.gov/Built-Environment/Building-Energy-Benchmarking-Data-2015-Present/teqw-tu6e/about_data

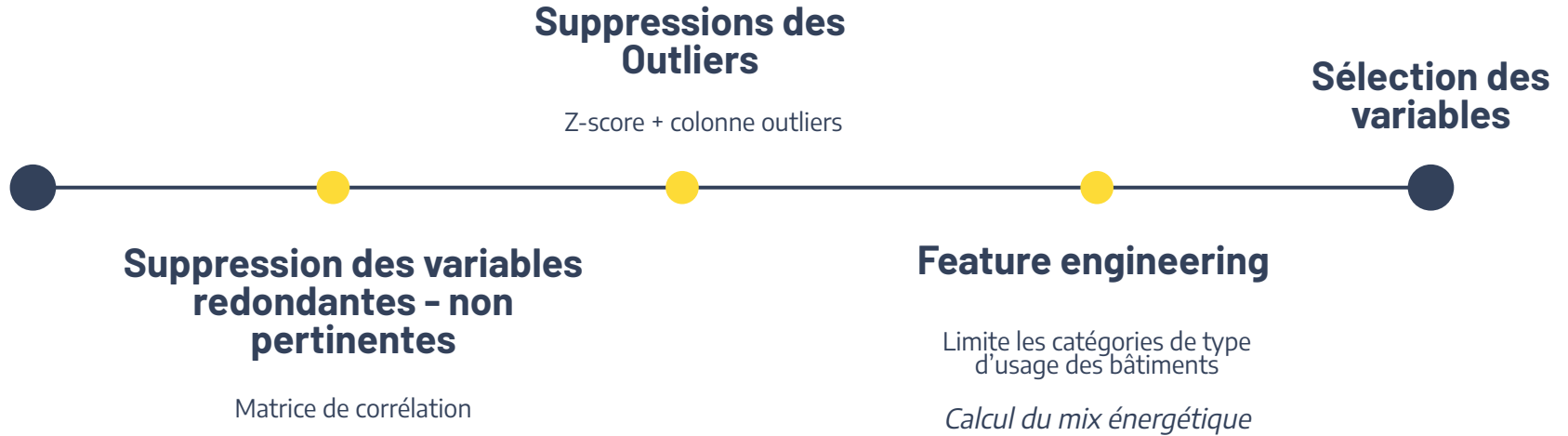
Variables catégorielles :
barplot du nombre de lignes par
valeur



Variable numérique :
Distribution
boxplot



Méthodologie



Informations disponibles	Modifications
Types de bâtiments	Suppression des bâtiments résidentiels (on ne garde que les bâtiments non résidentiels).
Localisation : adresse, latitude, longitude	Conservation uniquement de latitude et longitude.
Année de construction	Conservée
Nombre de bâtiments et d'étages	Conservée
Surfaces (Bâtiments, parking, total)	Conservation uniquement de la surface des bâtiments.
Usage des bâtiments	Conservation uniquement du PrimaryPropertyType Feature engineering ; <i>Certaines catégories sont peu représentées ou redondantes. Pour simplifier le modèle et améliorer sa robustesse, j'ai regroupé les catégories similaires par nature.</i>
Performances énergétiques	Supprimé : variables déjà dérivées de nos cibles.
Consommations énergétiques (kBtu, kWh, therms, gaz, electricity)	On ne garde les caractéristiques en kBtu *kBtu = kilo British thermal unit → unité d'énergie Feature engineering : <i>On transforme les consommations par type d'énergie en pourcentage du total pour représenter le mix énergétique sans révéler la consommation totale.</i>
Émissions GES : TotalGHGEmissions, GHGEmissionsIntensity	Conservation uniquement de TotalGHGEmissions.
Variables “autres” : Comments, DefaultData, ComplianceStatus, Outlier	Utilisées temporairement pour filtrer les bâtiments incorrects → supprimées ensuite.

Selection des données

1

Variables cibles

SiteEnergyUse(kBtu)	<i>consommation d'énergie totale</i>
TotalGHGEmissions	<i>Emissions de gaz à effet de serre.</i>

Taille de la BDD

3376 lignes - 46 colonnes



1292 lignes - 10 colonnes.

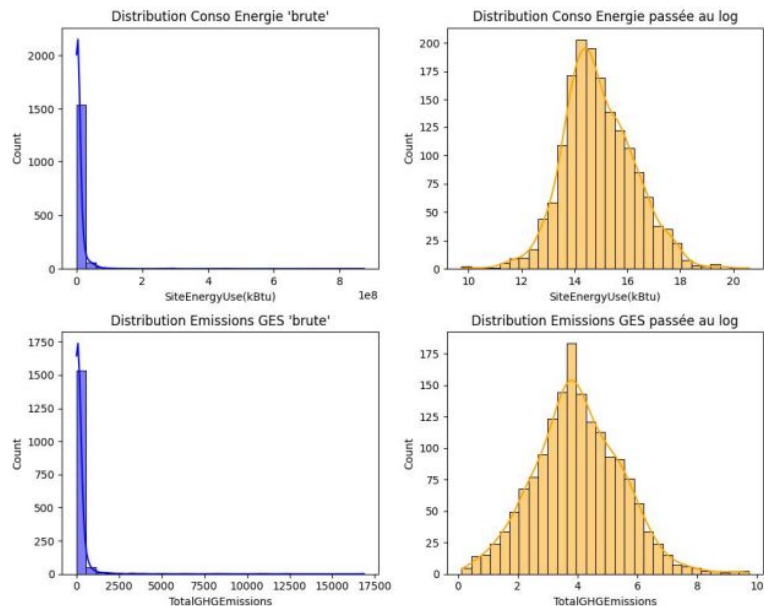
2

Variables explicatives (10)

PrimaryPropertyType	<i>Type principal du bâtiment.</i>
Latitude / Longitude	<i>Localisation géographique.</i>
YearBuilt	<i>Année de construction.</i>
NumberOfBuildings	<i>Nombre total de bâtiments présents sur le site.</i>
PropertyGFABuilding(s)	<i>Surface totale des bâtiments (GFA).</i>
pct_electricity / pct_steam (nouveaux attributs créés)	<i>Proportions des différents types d'énergie dans la consommation totale (électricité / gaz / vapeur).</i>

Data cleaning et analyse exploratoire

Le passage au log des variables cibles rapproche leur distribution de celle d'une loi normale :



3. Modélisation

Méthodologie

Séparer le jeu de données

Features -targets

Train-test split

Séparation variable entraînement

Choix du modèle

Encodage de la variable catégorielle

PrimaryPropertyType

Méthode One-Hot

Normalisation

Mettre à la même échelle - La normalisation rend chaque variable comparable (moyenne = 0, écart-type = 1).

Modélisation

	Model	MAE (M kBtu)	RMSE (M kBtu)	R2	R2 CV (5-fold)
0	LinearRegression	2.16	4.97	0.444	0.504
1	random forest	1.65	2.70	0.514	0.572
2	GradientBoosting	1.58	2.64	0.590	0.601
3	DummyRegressor	2.49	4.13	-0.028	-0.008

On peut comparer les modèles sur ces critères :

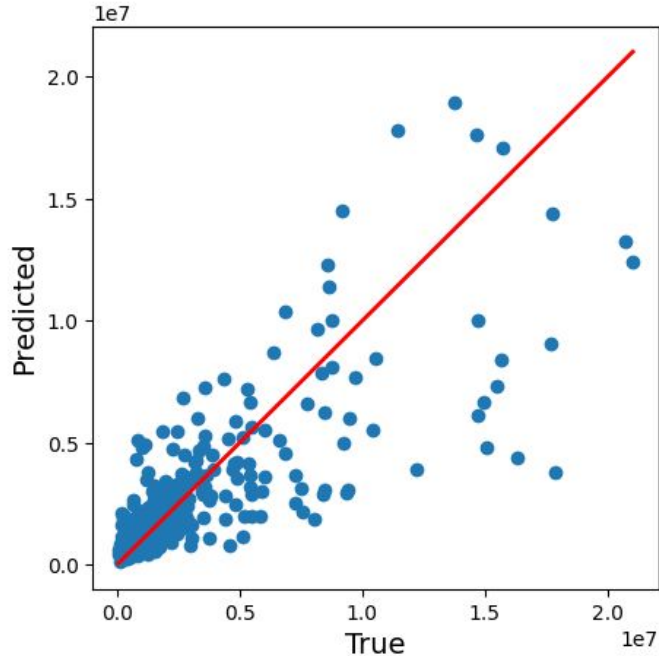
- **MAE (Mean Absolute Error)** : plus c'est petit, mieux le modèle prédit en moyenne.
- **RMSE (Root Mean Squared Error)** : plus c'est petit, mieux le modèle gère les grosses erreurs.
- **R²** : plus c'est proche de 1, mieux le modèle explique la variance.

Le GradientBoosting a le R² légèrement plus élevé et le MAE le plus bas, donc il explique un mieux la variance globale.

C'est donc le meilleur modèle pour notre prédiction.

Modélisation

Evaluation visuelle des performances du modèles après test d'optimisation avec GridSearch



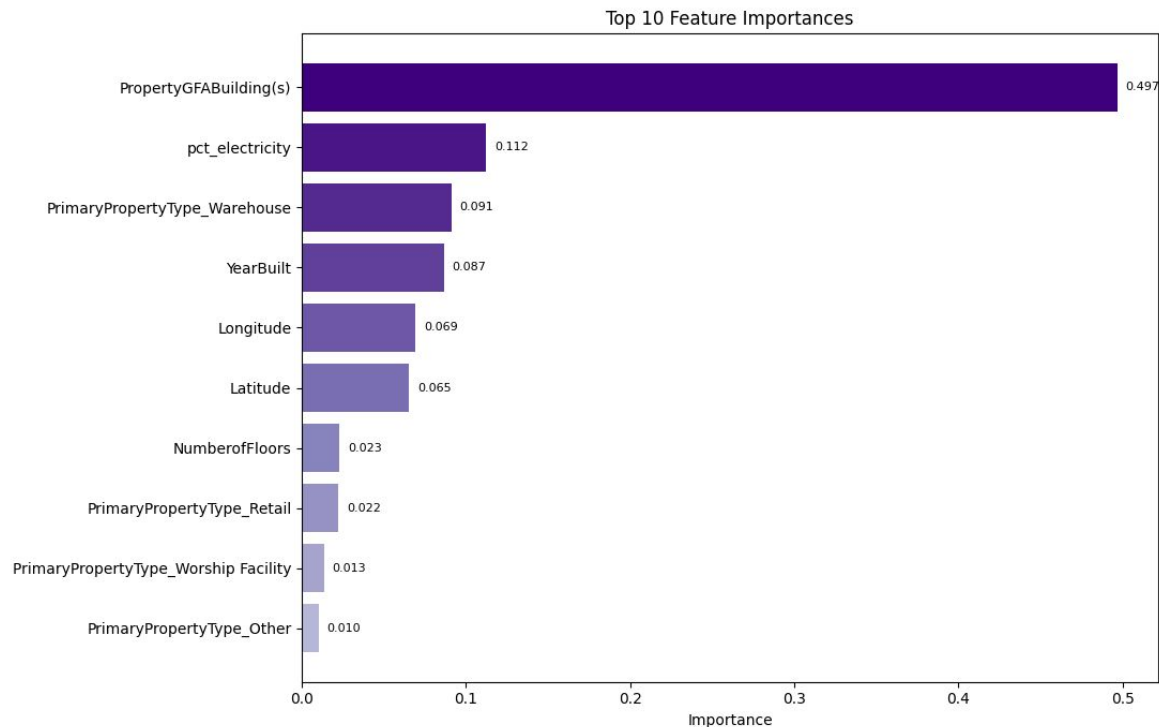
R^2 : Le coefficient de détermination est de 0.59 sur le test.

Cela signifie que le modèle explique environ 59 % de la variance de la variable cible.

Si notre modèle prédisait parfaitement la consommation énergétique des bâtiments, les valeurs prédites seraient égales aux valeurs réelles et se situeraient exactement sur la diagonale rouge

Modélisation

Les variables les plus impactantes via la méthode 'feature_importances'

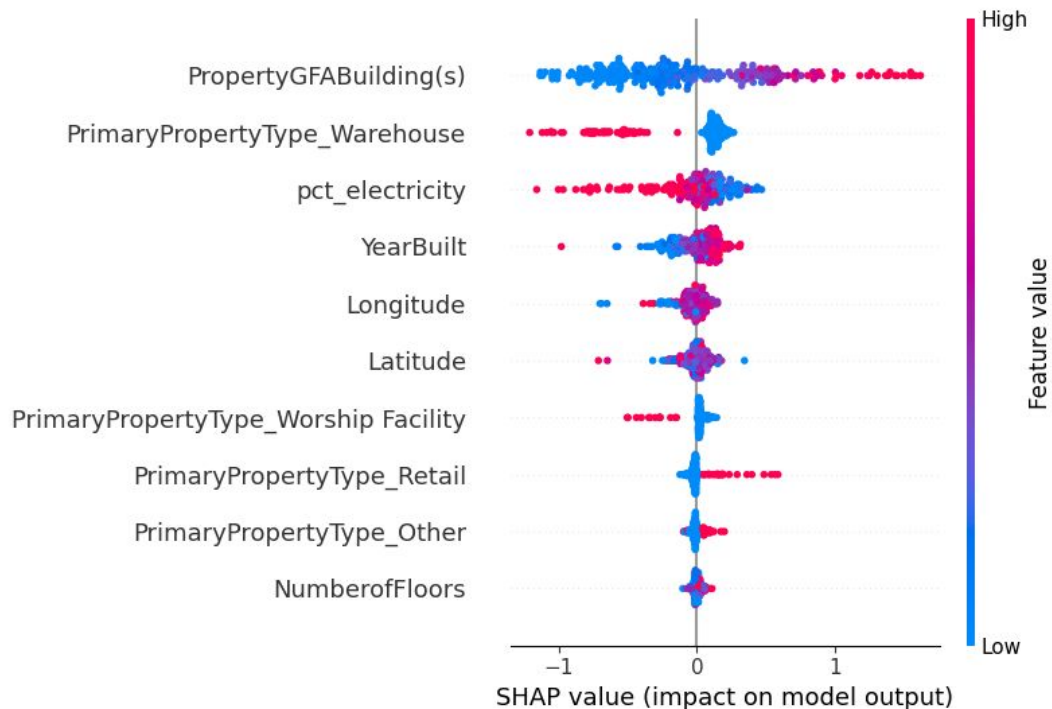


PropertyGFABuilding(s) : 50 % de l'importance totale
→ C'est la feature la plus influente.

pct_electricity, YearBuilt : environ 9–11%
- effet significatif

Type d'usage aussi à un impact (+ warehouse)

Modélisation



Ce graphique nous permet de détecter les relations existantes entre nos variables et la target.

la SHAP value évalue l'effet de chaque variable sur la prédiction : si l'influence des variables (rouge pour valeurs élevées et bleu pour les valeurs faibles) sur la cible est **positive ou négative**.

Si on regarde la variable « surface (GFA) », on voit que lorsqu'elle est principalement élevée (rouge), la valeur l'influence sur la consommation d'électricité positive. Cela signifie qu'une **surface plus élevée a tendance à influencer positivement le résultat**.

Surprenamment, il nous apprend que plus le bâtiments est récents, plus il est consommateur d'énergie. (à explorer !)

4. Création d'une API

Méthodologie

Création d'un service.py

Définition du schéma d'entrée
(variable à entrer, chargement du modèle, définition
des endpoints,
avec la librairie Pydantic: préciser les règles de
données à entrer)

Enregistrement du modèle

```
bentoml.sklearn.save_model("best_rf_model", best_model)
```

Test sur Swagger

Commande : `bentoml serve service:EnergyAPI --reload`

POST /predict

Parameters

No parameters

Request body

```
{
  "input_data": {
    "PrimaryPropertyType": "Office",
    "Latitude": 47.6,
    "Longitude": -122.3,
    "YearBuilt": 2000,
    "NumberofFloors": 5,
    "PropertyGFABuilding s": 10000,
    "pct_electricity": 0.7,
    "pct_steam": 0.3
  }
}
```

Server response

Code	Details
------	---------

200	
-----	--

Response body

```
{
  "prediction": 10345537.625931371
}
```

Execute

5. Déploiement sur le cloud

Méthodologie

poetry.lock

verrouille les versions exactes des packages Python pour garantir que l'environnement soit identique partout.

Création bentofile.yaml

définit le service, les fichiers à inclure, et les dépendances Python pour que BentoML construise l'image correctement.

docker image

contient ton API et ses dépendances, prête à tourner dans n'importe quel conteneur ou cloud.
Créé avec la commande "bentoml build"

- qui a lu bentofile.yaml,
- préparé le service,
- installé les dépendances,
- packager le modèle,
- puis généré une image Docker automatiquement.

```
(.venv) PS C:\Users\mathi\OneDrive\Documents\8_OCR\Projet_6\projet6_folder> $body = @{{
>>     input_data = @{
>>         PrimaryPropertyType = "Office"
>>         Latitude = 47.6
>>         Longitude = -122.3
>>         YearBuilt = 2000
>>         NumberofFloors = 5
>>         PropertyGFABuilding_s = 10000
>>         pct_electricity = 0.7
>>         pct_steam = 0.3
>>     }
>> } | ConvertTo-Json -Depth 3
>> Invoke-RestMethod -Uri http://localhost:3000/predict -Method POST -Body $body -ContentType "application
n/json"
>>
```

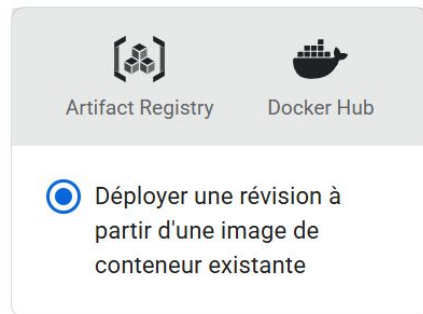
```
prediction
-----
10345537,625931373
```

```
(.venv) PS C:\Users\mathi\OneDrive\Documents\8_OCR\Projet_6\projet6_folder>
```


Artifact Registry API

L'**Artifact Registry API** est le service de Google Cloud : C'est un "dépôt d'images Docker"

- Se logger dans gcr
- tagger l'image locale
- push l'image docker



↪ URL de l'image du conteneur _____

```
PS C:\Users\mathi\OneDrive\Documents\8_OCR\Projet_6\projet6_folder> docker tag energyapi_test:7nv4
h26mpwc5boa6 gcr.io/ocr-projet6-api-479617/energyapi:latest
>>
PS C:\Users\mathi\OneDrive\Documents\8_OCR\Projet_6\projet6_folder> docker push gcr.io/ocr-projet6
-api-479617/energyapi:latest
The push refers to repository [gcr.io/ocr-projet6-api-479617/energyapi]
737ff14b014c: Pushed
3605883d7c67: Pushed
33088e098b09: Pushed
```

Cloud RUN : déployer l'API

Création du service [^ Masquer l'état](#)



Création du service ✓ Terminé(e)(s)

Création de la révision ✓ Terminé(e)(s)

Routage du trafic ✓ Terminé(e)(s)



energyapi

Région : europe-west1

URL : <https://energyapi-957289954604.europe-west1.run.app>



Scaling : auto

Observabilité

Révisions

Source

Déclencheurs

Réseau

Sécurité

YAML

Filtrer Filtrer les révisions



Nom	Trafic	Déploiement effectué ↓	Tags de révision	Actions
✓ energyapi-00001-nqf	100 % (vers la révision la plus récente)	il y a 1 minute	+	⋮ >

6. Fin projet

Axe d'amélioration et d'exploration

- Amélioration de l'API : welcome page - description
- Contrôle des coûts : comment être vigilant ?
- Comment télécharger un dataset complet pour des prédictions ?
Explorer la librairie Pandera.